

Backpropagation, Chain Rule and Gradient Flow

Introduction

In the previous stages of neural network learning, we learned that a neural network contains weights and biases that determine its predictions.

However, simply having weights is not enough. The network needs a mechanism to learn the appropriate values of these weights.

This is where backpropagation comes in.

Suppose we give an image of a cat to a neural network. Initially, the network may predict:

Cat: 30%

Dog: 70%

The prediction is incorrect. The network needs to understand:

- How wrong was the prediction?
- Which parameters contributed to the error?
- Which parameters should be increased?
- Which parameters should be decreased?
- By how much should they change?

Backpropagation provides the information needed to answer these questions.

What is Backpropagation?

Backpropagation is an algorithm used to calculate the gradients of a neural network's loss with respect to its weights and biases.

In simpler words:

Backpropagation tells the neural network how its parameters contributed to the prediction error.

The process consists of two major directions:

Forward direction

Information moves from: Input→Hidden Layers→Output

Backward direction

Error information moves from: Loss→Output Layer→Hidden Layers

Why Do We Need Backpropagation?

Consider a neural network with three layers.

Input



Layer 1



Layer 2



Layer 3



Output



Prediction

Suppose the prediction is wrong.

The network needs to adjust the parameters of all three layers.

But the parameters in Layer 1 do not directly produce the final prediction. Their effect passes through Layer 2 then Layer 3 and then the output layer.

For example:

Weight in Layer 1



Neuron output



Layer 2



Layer 3



Final prediction



Loss

Therefore, we need to determine how a change in the Layer 1 weight eventually affects the final loss.

This is the fundamental problem solved by backpropagation.

Forward Propagation

Before discussing the backward pass, we need to understand the forward pass.

During forward propagation, the input travels through the network.

For a simple neuron:

$$z = wx + b$$

where:

- x = input
- w = weight
- b = bias
- z = weighted input

Then an activation function is applied:

$$a = f(z)$$

The output a is passed to the next layer.

For example:

Input



Weighted Sum



Activation Function



Output

This process continues until the network produces its final prediction.

Calculating the Loss

After the network produces a prediction, we need to determine how good or bad that prediction is.

This is the purpose of the loss function.

For example, suppose:

Actual value = 1

Predicted value = 0.7

The prediction is not perfect, so the loss function produces an error value.

The exact loss function depends on the problem.

Common examples include:

- Mean Squared Error (MSE)
- Binary Cross-Entropy
- Categorical Cross-Entropy

For our discussion, we don't need to focus heavily on the mathematics of the loss function.

The important idea is:

The loss tells us how well the network performed.

A smaller loss generally means a better prediction.

What Happens After Calculating the Loss?

Now the network knows:

"My prediction was not correct enough."

But this information alone is not sufficient.

The network must determine:

Which parameters caused the error?

This is where backpropagation begins.

The error information is propagated backward through the network.

Loss



Output Layer



Hidden Layer



Earlier Hidden Layer



Parameters

At every layer, backpropagation determines how much that layer contributed to the final error.

The Chain Rule

The chain rule is the mathematical foundation of backpropagation.

The basic idea is simply:

When one quantity affects another quantity through several intermediate steps, we can determine the overall effect by combining the effects at each step.

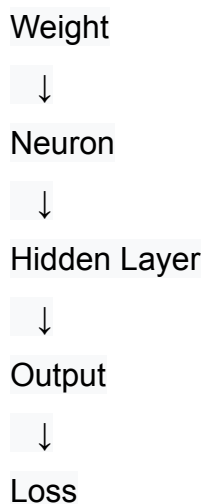
Consider:

$$A \rightarrow B \rightarrow C \rightarrow D$$

If A changes B, B changes C, and C changes D, then a change in A eventually affects D.

The chain rule allows us to trace this effect.

In a neural network:



The chain rule allows us to determine:

How much did that weight ultimately affect the loss?

A Small Mathematical Example

Consider:

$$z=wx+b$$

followed by an activation function:

$$a=f(z)$$

and finally a loss:

$$L=\text{Loss}(a,y)$$

We want to know:

$$\partial w / \partial L$$

Instead of calculating it all at once, we break it into smaller steps:

$$\partial w / \partial L = \partial a / \partial L \times \partial z / \partial a \times \partial w / \partial z$$

This is the chain rule.

The important idea is:

Backpropagation breaks a complicated calculation into smaller local calculations and connects them using the chain rule.

What is a Gradient?

A gradient tells us how much the loss changes when a particular parameter changes.

Suppose we have a weight w .

Its gradient is:

$$\partial w / \partial L$$

We can interpret this as:

"If I change this weight slightly, how will the loss change?"

The gradient has two important pieces of information:

Magnitude

How strongly does the parameter affect the loss?

Direction

Should the parameter move upward or downward to reduce the loss?

Simple Interpretation of a Gradient

Suppose two weights have gradients:

Weight A $\rightarrow$ gradient = large

Weight B $\rightarrow$ gradient = small

This means that Weight A has a stronger immediate influence on the loss than Weight B.

Therefore, the optimizer may make a larger adjustment to Weight A.

This allows different parameters to be updated by different amounts.

Backpropagation and Gradient Descent

Backpropagation and gradient descent are related, but they are not the same thing.

Backpropagation

Calculates the gradients.

Gradient Descent

Uses those gradients to update the parameters.

The process is:

Prediction



Loss



Backpropagation



Gradients



Gradient Descent / Adam



Updated Weights

A simple gradient descent update is:

$$w_{\text{new}} = w_{\text{old}} - \eta(\partial w / \partial L)$$

where:

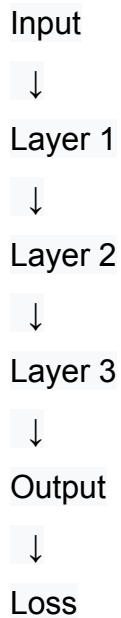
- w = weight
- η = learning rate
- $\partial w / \partial L$ = gradient

Backpropagation calculates; the optimizer updates.

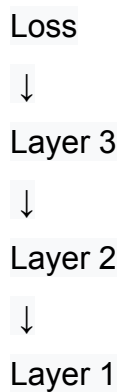
Gradient Flow

Gradient flow refers to the movement of gradient information backward through the network during backpropagation.

Consider:



The gradient flows in the opposite direction:



Every layer uses the incoming gradient to determine how its own parameters should be adjusted.

Why Gradient Flow Matters

Imagine a network with 100 layers.

The gradient has to travel backward through many layers.

Loss



Layer 100



Layer 99



Layer 98



...



Layer 2



Layer 1

If the gradient is preserved properly, the early layers can learn.

But if the gradient becomes too small or too large while traveling backward, training becomes difficult.

This gives us two important problems:

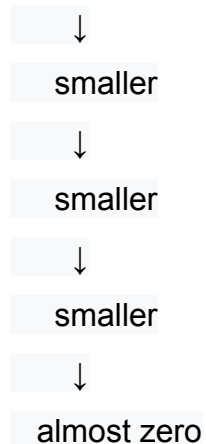
- Vanishing gradient
- Exploding gradient

Vanishing Gradient Problem

The vanishing gradient problem occurs when gradients become extremely small as they move backward through a deep network.

Conceptually:

Gradient at output



Eventually, the early layers receive almost no useful learning signal.

As a result:

- early layers learn very slowly,
- training becomes inefficient,
- the network may fail to learn useful representations.

Why Does the Gradient Vanish?

During backpropagation, gradients are repeatedly transformed as they pass through different layers.

If each transformation reduces the gradient slightly, the effect accumulates over many layers.

For example, imagine a gradient is repeatedly reduced by half:

A vertical sequence of numbers and arrows illustrating the vanishing gradient problem. It starts with the number 1, followed by a downward arrow, then 0.5, another downward arrow, then 0.25, a third downward arrow, then 0.125, a fourth downward arrow, then 0.0625, a fifth downward arrow, and finally an ellipsis (...).

1
↓
0.5
↓
0.25
↓
0.125
↓
0.0625
↓
...

After many layers, it can become extremely small.

This is the basic intuition behind the vanishing gradient problem.

Exploding Gradient Problem

The opposite problem is the exploding gradient problem.

Here, the gradient becomes increasingly large as it moves backward.

Conceptually:

Gradient



Larger



Much larger



Extremely large

This can result in:

- very large weight updates,
- unstable training,
- numerical problems,
- failure to converge.

Therefore, a good neural network needs a healthy gradient flow.

Role of Activation Functions

Activation functions have a significant impact on gradient flow.

Common activation functions include:

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- GELU

Older networks frequently used sigmoid and tanh.

However, sigmoid can produce very small gradients in certain regions.

This can make gradient propagation difficult in deep networks.

ReLU

One of the most widely used activation functions is ReLU.

Its basic behavior is:

Input $< 0 \rightarrow$ Output = 0

Input $> 0 \rightarrow$ Output = Input

ReLU became popular partly because it generally allows better gradient propagation for positive activations than sigmoid.

However, it can also suffer from the dying ReLU problem, where some neurons can become inactive and stop contributing useful gradients.

How Modern Networks Improve Gradient Flow

Modern deep learning architectures use several techniques to make training deep networks easier.

Some important techniques include:

1. Better activation functions

Examples:

- ReLU
- Leaky ReLU
- GELU

2. Proper weight initialization

Weights need to be initialized appropriately so that activations and gradients do not become excessively large or small.

3. Normalization

Techniques such as Batch Normalization and Layer Normalization can help stabilize training.

4. Residual connections

Used famously in ResNet, residual connections provide shortcut paths through the network.

5. Appropriate optimization algorithms

Methods such as Adam can make parameter updates more effective.

Backpropagation in CNNs

Backpropagation is especially important for Convolutional Neural Networks.

Consider an image classification network:

Image



Convolution



Activation



Pooling



Convolution



Fully Connected Layer



Prediction



Loss

During backpropagation:

Loss



Fully Connected Layer



Convolution



Pooling



Earlier Convolution

The gradients are used to update the convolution filters and other parameters.
Over many training iterations, these filters learn useful visual features.

Example: Image Classification

Suppose we are training a CNN to classify brain CT images into:

- Normal
- Ischemic stroke
- Hemorrhagic stroke

Initially, the model may make many incorrect predictions.

The training process is:

Step 1: Input image

The CT image enters the network.

Step 2: Forward propagation

The CNN extracts features and produces a prediction.

Step 3: Calculate loss

The prediction is compared with the correct class.

Step 4: Backpropagation

The error information is propagated backward.

Step 5: Calculate gradients

The network determines how its filters and other parameters contributed to the error.

Step 6: Update parameters

The optimizer adjusts the parameters.

Step 7: Repeat

This process occurs for many images and many training epochs.

Eventually, the CNN learns increasingly useful features for distinguishing the classes.

Backpropagation and Feature Learning

One of the most important advantages of neural networks is that they can learn features automatically.

For a CNN, the hierarchy may look conceptually like:

Early Layers



Edges and simple patterns



Middle Layers



Textures and shapes



Deep Layers



Complex structures



Classification

Backpropagation provides the learning signal that allows these features to improve during training.

A Simple Analogy

Consider a student solving a mathematics problem.

The student gives an answer.

The teacher says:

"Your answer is incorrect."

But simply saying "wrong" isn't enough.

The student needs feedback:

"This step was correct, but this step caused the mistake."

The student then goes back through the solution and corrects the problematic steps.

Backpropagation works in a similar conceptual way.

Network makes prediction



Prediction is wrong



Find the source



Propagate feedback backward



Adjust parameters



Try again

Backpropagation: Complete Conceptual Picture

The complete learning cycle can be summarized as follows:

Forward Phase

Input → Network → Prediction

The network uses its current parameters to make a prediction.

Loss Calculation

Prediction → Loss

The loss function determines how far the prediction is from the expected result.

Backward Phase

Loss → Network

Backpropagation determines how the parameters contributed to the loss.

Optimization Phase

Gradients → Parameter Updates

The optimizer modifies the parameters.

Repeat

The process continues until the model learns a useful representation.

Important Terminology

Term	Meaning
Forward Propagation	Passing input through the network to produce an output
Loss	Measure of how incorrect the prediction is
Backpropagation	Calculating gradients by propagating information backward
Chain Rule	Mathematical rule used to calculate the effect through multiple operations
Gradient	Indicates how the loss changes with respect to a parameter
Gradient Flow	Movement of gradients backward through the network
Optimizer	Uses gradients to update model parameters
Learning Rate	Controls the size of parameter updates
Vanishing Gradient	Gradient becomes extremely small
Exploding Gradient	Gradient becomes extremely large
Residual Connection	Shortcut connection that helps information and gradients flow